
name: autonomous-engine-site description: Use when building a website that doubles as an autonomous growth engine — a site that not only presents the business but continuously finds prospects who need the service, reaches out with a tangible deliverable, captures responses, and converts, all on a schedule with minimal human input. Trigger whenever the user wants a “self-running site,” “growth engine,” “autonomous outreach app,” “lead-gen site,” or a website that “pulls users in and offers itself.” This skill specifies the COMPONENTS to build and how to OPTIMISE the site. It is deliberately tech-agnostic — it names the parts and their contracts, not the stack. Pick tools to fit.

Autonomous Engine Site

A blueprint for a website that is also a self-running acquisition machine. It specifies the components, how they connect, what configuration each needs, and how to optimise the whole. No stack is prescribed — every component is described by its responsibility and its inputs/outputs so any tooling can fill it.

Build in two layers: the **public site** (what visitors see) and the **engine** (the autonomous loop running behind it). They share one data spine.

Layer 1: The public site — components

The site is the engine's landing surface and conversion endpoint. Every component below should exist.

1. **Entry / landing component.** Communicates within seconds what the service is, who it's for, and the single next action. One dominant call-to-action above the fold. Variant-

aware: it can render a personalized version when a visitor arrives from an outreach link (see Personalization token).

2. **Value-proof component.** The credibility surface — proof, results, named references, sample deliverables. This is what converts a curious click into a lead. Make it modular so new proof drops in without a rebuild.
3. **Deliverable / artifact component.** The page where a prospect receives or requests the tangible thing the engine produces for them (audit, report, prototype, guide). This is the heart of the conversion: the site *gives* before it asks. Supports both a generic version (for cold visitors) and a personalized version keyed to an outreach token.
4. **Capture component.** Forms and intent-capture that feed the data spine. Minimum friction: ask only for what the next action needs. Every capture writes a lead record and fires an event into the engine.
5. **Conversion / booking component.** The defined next step — schedule, subscribe, start engagement. This is the engine's exit. It must be one obvious click from the deliverable, never buried.
6. **Personalization token system.** A mechanism where outreach links carry an identifier so the site greets a known prospect with their context (their company, the trigger that flagged them, a deliverable built for them). This single component is the biggest single conversion lever the site has.
7. **Consent + preference component.** Visible privacy controls, tracking consent, and a working unsubscribe/opt-out that writes back to the suppression list. Legally required and trust-building.
8. **Analytics + event surface.** Every meaningful interaction (view, scroll-depth, capture, booking, deliverable open) emits an event the engine can learn from. Without this the loop is blind.

Layer 2: The autonomous engine — components

The loop that runs on a schedule and feeds the site with qualified, warmed prospects. Each component is a stage with a clear input and output; chain them.

1. **Signal-detection component.** Continuously scans defined sources for prospects who show *need + timing*. Input: source feeds. Output: candidate records with a fit/timing score. Configuration needed: the source list, the scoring rules, the scan schedule, dedup keys.

2. **Enrichment component.** Takes a candidate and attaches the context needed to personalize: the triggering signal, the right contact, the likely pain. Input: candidate record. Output: enriched profile. Configuration: which fields are required before a profile can advance.
3. **Qualification + compliance gate.** Filters hard — most candidates drop here. Scores fit, timing, reachability, and legal eligibility (consent/jurisdiction). Input: enriched profile. Output: approved prospect OR drop/hold. Configuration: pass thresholds, the jurisdiction/consent ruleset, and the global suppression-list check. **This gate is mandatory and non-bypassable** — it protects deliverability and keeps the system lawful.
4. **Artifact-generation component.** Produces the tangible deliverable that solves a slice of the prospect's specific problem, tiered by prospect value:
 - *Broad tier:* an automatable custom audit/report built from the data that flagged them.
 - *Mid tier:* a working partial solution configured with their data.
 - *Priority tier:* a physical, personalized piece dispatched by post for high-value targets. Input: enriched profile. Output: a rendered artifact + a unique personalization token linking it to a site page. Configuration: the tier rules, the templates, the per-tier effort budget. Design rule: the artifact must be genuinely useful even if they never reply. A "gift" that is really a pitch reads as spam.
5. **Delivery component.** Sends the artifact through the right channel with the right infrastructure. Input: artifact + token + contact. Output: a tracked delivery + a delivery event. Configuration: channel, sending identity (separate from the primary domain), volume caps, footer/identity, retry rules. Principle: precision over volume — fewer, sharper touches outperform mass sends and protect reputation.
6. **Response-handling component.** Classifies replies and site events (interested / not / objection / unsubscribe) and routes them. Interested → the site's conversion component or a human. Unsubscribe → suppression list, instantly and permanently. Input: inbound replies + site events. Output: routed action + updated lead state.
7. **Feedback / learning component.** Tracks which signals and which artifact angles actually convert and feeds that back into scoring (component 1) and generation (component 4). Input: outcome events. Output: updated scoring weights and template-performance data. This is what makes the engine sharper over time.
8. **Orchestration component.** Runs the loop on a schedule and passes state between every stage. Input: the schedule + the shared data spine. Output: the loop executing daily/continuously without manual triggering. Configuration: cadence, per-stage

concurrency, failure handling, alerting.

9. **Shared data spine.** The single store every component reads and writes: prospects, profiles, artifacts, tokens, events, suppression list, outcomes. The site and the engine both bind to this. One source of truth — no component keeps private state the others can't see.
-

Required configurations checklist

Whatever the stack, these must be configured for the engine to run safely and well:

- Source list + scan schedule for signal detection
 - Fit/timing scoring rules and qualification thresholds
 - Jurisdiction/consent ruleset + global suppression list (wired into the gate AND the site's opt-out)
 - Artifact tier rules, templates, and per-tier effort/cost budgets
 - Personalization token scheme shared between engine and site
 - Sending identity separate from the primary domain, with volume caps and warm-up
 - Event schema emitted by the site and consumed by the learning component
 - Orchestration cadence, concurrency, retry, and alerting
 - Analytics + conversion tracking end-to-end (outreach link → site → booking)
 - Backup/audit log of who was contacted, with what, and why
-

How to optimise the site

Optimisation runs on two fronts: the build quality bar, and the conversion/learning loop.

Build quality bar (table stakes — clear all of these)

- **Speed:** Largest Contentful Paint under ~2.5s, Interaction to Next Paint under 200ms, Cumulative Layout Shift under 0.1.
- **Mobile-first:** design from the phone up — responsive, tap-friendly, no horizontal scroll, readable without zoom.

- **Accessible:** WCAG — contrast, keyboard navigation, alt text, semantic structure, screen-reader support.
- **Secure & private:** HTTPS everywhere, sane data handling, explicit tracking consent.
- **Findable:** clean semantic structure, structured data/schema markup, fast indexing, legibility to AI systems that summarize and cite pages.
- **Clear purpose:** a visitor knows in seconds what it is and what to do next; one dominant call-to-action.
- **Lightweight:** modern image formats, lazy loading, minimal unnecessary scripting.

Conversion + learning optimisation (the engine's compounding edge)

- **Personalize the landing on arrival.** A page that reflects the prospect's own context (via the token) converts far above a generic one. This is the highest-leverage optimisation.
- **Lead with the deliverable, ask second.** The site gives value before requesting anything; the conversion step is one click from the artifact.
- **Shorten the path to the next action.** Remove every field, page, and decision that isn't required to convert. Measure path length and cut it.
- **Instrument everything and close the loop.** Every view, scroll, open, and booking feeds the learning component so scoring and artifacts improve automatically. An engine that doesn't learn from site behaviour is half an engine.
- **Test variants continuously.** Headline, proof, artifact framing, call-to-action — run variants and let outcome data pick winners; feed those back into outreach copy too.
- **Match the site to the outreach.** The artifact a prospect received and the page they land on must feel like one continuous experience; mismatch kills conversion.
- **Protect speed as you add components.** Each new tracker or widget costs load time; budget it. A fast generic page beats a slow personalized one.

Sequence to optimise in

1. Get the build quality bar green first — nothing converts on a slow, unclear, inaccessible site.
2. Wire the token personalization and the deliverable-first flow — the biggest conversion gains.
3. Instrument the event surface and connect the learning loop — so the system compounds.

4. Then iterate variants continuously on outcome data.
-

Build order

1. Stand up the **shared data spine** — every component depends on it.
2. Build the **public site** components to the quality bar, with the token system and event surface wired in.
3. Build the **engine** stages, ending with **orchestration** so the loop runs on schedule.
4. Connect the **learning component** both ways (site events in, scoring/templates out).
5. Configure everything on the checklist, verify the **compliance gate and opt-out** end to end, then let the loop run and optimise on the data it returns.